

系统开发工具基础

第 4 周课后实验报告

姓名：彭俊皓 学号：25020007100

2026 年 9 月 14 日

仓库：<https://github.com/pjh456/sys-devtool/>

本报告覆盖第 4 周课堂四道实验（week4/q13–q16）的操作过程与结果，以及 missing-semester 三讲课后练习：《版本控制》的历史重写、stash、别名与全局 gitignore（homework/version-control/）、《代码质量》的 CI 格式门禁与正则（homework/code-quality/）、《不止于代码》的提交信息与代码评审（homework/beyond-code/）；全部结果均在本机重新执行验证。

表 1: 第 4 周课后实验完成情况

部分	内容	关键工具	状态
一	课堂实验回顾（q13–q16）	ruff、pytest、make、curl、jq	✔ 完
二	课后：《版本控制》历史重写/stash/别名/gitignore	git-filter-repo、stash、config	✔ 完
三	课后：《代码质量》CI 门禁与正则	clang-format、GitHub Actions、grep、sed、re	✔ 完
四	课后：《不止于代码》提交信息与评审	git、gh、Markdown	✔ 完

一

课堂实验回顾（q13–q16）

q13

建立可执行的本地质量门禁

📖

题目要求

- 复制 q10 为 q13，在 pyproject.toml 加入 ruff 与 pytest 的最小配置；
- 补充一个正常姓名测试与一个空白姓名测试，每个测试包含有效断言；
- 运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则；
- 编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest，保证返回 0。

</>

质量配置（q13/pyproject.toml）

1

[tool.pytest.ini_options]

```
2 pythonpath = ["src"]
3 testpaths = ["tests"]
4
5 [tool.ruff]
6 target-version = "py312"
7 line-length = 88
```

🔗 测试 (q13/tests/test_cli.py)

```
1 def test_normal_name(monkeypatch, capsys):
2     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "alice"])
3     main()
4     assert capsys.readouterr().out == "Hello, alice!\n"
5
6 def test_blank_name(monkeypatch, capsys):
7     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])
8     with pytest.raises(SystemExit) as exc:
9         main()
10    assert exc.value.code == 2
11    assert "must not be blank" in capsys.readouterr().err
```

🔗 门禁脚本 (q13/check.sh)

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 cd "$(dirname "$0")"
4
5 ruff format --check .
6 ruff check .
7 pytest
```

📌 结果与验证

```
$ ./check.sh
3 files already formatted
All checks passed!
tests/test_cli.py .. [100%]
2 passed in 0.01s
$ echo $?
0
```

三个工具串成一条门禁，任一失败即非零退出 (set -euo pipefail)。

💡 要点与注意事项

ruff format --check 只报告不修改，适合 CI；pytest 通过 pyproject.toml 的 [tool.pytest.ini_options] 读取 pythonpath 与 testpaths，无需额外命令行参数。

q14 让 Make 只重建真正受影响的产物**📖 题目要求**

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md;
- 编写 Makefile, 含 all、stats.txt、report.txt 和 clean, 准确列出数据、脚本与中间产物依赖, 并把 all、clean 声明为 .PHONY;
- 验证首次 make 生成两个产物、第二次无改动不执行配方、touch data.csv 后两个产物重新生成、clean 只删生成物。

🔗 Makefile (q14/Makefile)

```
1 .PHONY: all clean
2
3 all: stats.txt report.txt
4
5 stats.txt: data.csv stats.py
6     python3 stats.py
7
8 report.txt: stats.txt report.md build_report.py
9     python3 build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

✅ 结果与验证

```
$ make                # 首次: 两个产物都生成
python3 stats.py
python3 build_report.py
$ make                # 无改动: 不执行任何配方
make: 对"all"无需做任何事。
$ touch data.csv; make # 数据变更: 两个目标依次重建
python3 stats.py
python3 build_report.py
$ make clean
rm -f stats.txt report.txt
```

stats.txt 为 10 (2 + 3 + 5), report.txt 末尾为 Total: 10。

💡 要点与注意事项

正确性取决于依赖写全: stats.txt 依赖 data.csv 与 stats.py, report.txt 依赖 stats.txt、report.md 与 build_report.py; 少写任一依赖都会出现“改了源文件却不重建”。all/clean 不是文件, 必须 .PHONY, 否则同名文件会干扰目标判定。

q15 把本地 API 数据转换为可读报告**📖 题目要求**

- 保存给定 JSON 为 `packages.json`, 在本目录运行 `python -m http.server 8000`;
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`;
- 用 `jq` 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录, 并按 `downloads` 降序、`name` 升序排列;
- 编写 `api_report.sh` 生成 `summary.md`, 含标题与三列表格。

</> 报告脚本 (`q15/api_report.sh`)

```
1 curl -fsS "$URL" |
2   jq -r '
3     [ .[] | select(.status == "active" and .downloads >= 100) ]
4     | sort_by([-.downloads, .name])
5     | ([ "# Active Packages Report", "",
6         "| name | version | downloads |",
7         "| --- | --- | --- |",
8         + [ .[] | "| \(.name) | \(.version) | \(.downloads) |" ]
9         | .[]
10    ' > "$OUT"
```

✅ 结果与验证

```
$ bash api_report.sh
$ cat summary.md
# Active Packages Report

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

beta (inactive)、epsilon (80) 被过滤; delta 与 gamma 下载量同为 450, 按 name 升序把 delta 排在 gamma 前。

💡 要点与注意事项

`select` 负责过滤, `sort_by([-.downloads, .name])` 用“下载量取负”实现降序、同值再按名称升序; `curl -fsS` 让 HTTP 错误直接失败, 避免把错误页写进报告。

q16 修复并交付一个陌生的小型工具仓库

📄 题目要求

- 复制 q13 为 q16 并纳入 Git;
- 把问候语临时改成始终返回字面量 `Hello, name!`, 运行 `check.sh` 确认测试失败, 随后定位并修复;
- 编写含 `check`、`build`、`clean` 的最小 Makefile;
- 运行 `make check` 与 `make build`, 计算 wheel 的 SHA-256, 并把最终改动提交为一次聚焦提交。

🔗 Makefile 与构建校验 (q16/)

```
1 .PHONY: check build clean
2
3 check:
4     ./check.sh
5
6 build:
7     uv build
8
9 clean:
10    rm -rf dist/ src/*.egg-info
```

✅ 结果与验证

```
$ make check
3 files already formatted
All checks passed!
2 passed in 0.01s
$ make build
Successfully built dist/greetlab_25020007100-0.1.0-py3-none-any.whl
$ sha256sum dist/*.whl
2f6a84795591edb841788d44b901222f1ca4b1fdaed8ee37992160997fee4652 dist/...whl
```

错误版本把 `print(f"Hello, {a.name}!")` 改成字面量 `print("Hello, name!")`, `test_normal_name` 随即失败; 改回后门禁恢复全绿。

💡 要点与注意事项

字面量改动不会让程序崩溃, 只是输出错了——正是断言式测试能抓到而“能跑”抓不到的缺陷。wheel 不是字节级可复现的 (构建会写入时间戳等元数据), 同一源码重新构建得到的 SHA-256 可能不同, 因此哈希是“本次产物”的指纹, 而非源码的指纹。

二 课后练习: missing-semester 《版本控制》

版本控制·3 从历史中彻底删除一个误提交的文件

📖 题目要求

把一个文件加入仓库、做若干次提交，然后把它从历史中删除（不只是最新提交），可参考 GitHub 的移除敏感数据说明。

🔗 解答 (homework/version-control/3.sh)

```
1 repo=$(mktemp -d); cd "$repo"
2 git init -q
3 git config user.name demo; git config user.email demo@example.com
4 printf 'password=hunter2\n' > credentials.txt
5 printf 'public\n' > public.txt
6 git add . && git commit -qm "add public.txt and credentials.txt"
7
8 git filter-repo --path credentials.txt --invert-paths --force
9 ! git log --all --oneline -- credentials.txt | grep -q . # 历史中已不存在
```

✅ 结果与验证

```
$ sh 3.sh
Parsed 1 commits
New history written in 0.01 seconds; now repacking/cleaning...
Repacking your repo and cleaning out old unneeded objects
Completely finished after 0.02 seconds.
$ echo $?
0
```

`git filter-repo --invert-paths` 重写所有含该文件的提交，重写后 `git log --all -- credentials.txt` 无输出。

💡 要点与注意事项

单纯 `git rm` + 提交只删掉最新快照，历史里的旧版本仍可被检出；`filter-repo` 会重写提交（哈希全变）。若历史已推送，需 `force push` 并通知协作者重新克隆——这也说明敏感信息一旦进过仓库就应尽快轮换密钥，而不能只依赖删历史。

版本控制·4 git stash 的暂存与恢复

📖 题目要求

克隆一个仓库并修改某个文件，观察执行 `git stash` 后发生什么、`git log --all --oneline` 显示了什么；再 `git stash pop` 撤销；思考它适用的场景。

</> 观察 (homework/version-control/4.txt)

```
$ git stash
$ git log --all --oneline
6a99921 add note.txt
4e39d44 WIP on master: 6a99921 add note.txt
ac87543 index on master: 6a99921 add note.txt
$ git stash pop
```

✓ 结果与验证

stash 把未提交改动收起、工作区回到 HEAD; stash 保存在 refs/stash 上, 所以 git log --all 能看到 WIP on 与 index on 两条提交; stash pop 重新应用改动并从栈中删除。

💡 要点与注意事项

适用场景: 改到一半要临时切分支、拉取远端或修别处 bug, 先 stash 保持工作区干净, 处理完再 pop 回来。它只收未提交改动, 不改提交历史。

版本控制·5 为 git 添加 graph 别名

📋 题目要求

在 ~/.gitconfig 中创建别名 git graph, 使其等价于 git log --all --graph --decorate --oneline。

</> 解答 (homework/version-control/5.sh)

```
1 git config --global alias.graph 'log --all --graph --decorate --oneline'
```

✓ 结果与验证

```
$ git graph -6
* 8385384 (HEAD -> master) feat: 完成 不止于代码 06 题 PR 评审分析
* 22d5c09 feat: 完成 不止于代码 02 题提交信息对比
* 328e52c feat: 完成 代码质量 03 题 CI 加 clang-format 门禁
...
```

实际使用中该别名改由 dotfiles 以 XDG 全局配置管理: dotfiles/etc/.config/git/config (提交 ca82d3a, 部署到 ~/.config/git/config), ~/.gitconfig 不再单独保留该行, 便于随 dotfiles 一起版本化与迁移。

💡 要点与注意事项

`git config --global` 写的是 `~/.gitconfig`; Git 同时把 `~/.config/git/config` 当作全局配置读取, 后者更适合交给 `dotfiles` 管理。--all --graph --decorate --oneline 一行即可看清所有引用的分支/合并拓扑。

版本控制·6 配置全局 gitignore

📖 题目要求

运行 `git config --global core.excludesfile ~/.gitignore_global` 指定全局忽略文件的位置 (文件仍需手动创建), 写入忽略 OS/编辑器临时文件 (如 `.DS_Store`) 的规则。

🔗 解答 (homework/version-control/6.sh)

```
1 git config --global core.excludesfile "$HOME/.gitignore_global"
2
3 cat > "$HOME/.gitignore_global" <<'EOF'
4 .DS_Store
5 Thumbs.db
6 desktop.ini
7 *~
8 *.swp
9 *.sw0
10 EOF
```

✅ 结果与验证

```
$ git config --global --get core.excludesfile
/home/pjh123/.gitignore_global
$ git status --short --ignored
!! .DS_Store
!! desktop.ini
```

`.DS_Store`、`desktop.ini` 不再出现在普通 `git status` 中, `--ignored` 时才显式列出。本机验证后已回退 (`core.excludesfile` 清空、`~/.gitignore_global` 删除), 因为该需求不必要。

💡 要点与注意事项

`core.excludesfile` 是用户级忽略文件, 优先级最低 (低于仓库 `.gitignore` 与 `.git/info/exclude`), 且只影响未跟踪文件, 不会让已跟踪文件消失。

三 课后练习：missing-semester 《代码质量》

代码质量·3 为项目接入 CI 格式门禁

📖 题目要求

为一个正在进行的项目设置持续集成，让它在每次 push 时运行格式化、lint 与测试；故意把代码弄坏（如引入一条 lint 违规），确认 CI 能抓到。

🔗 新增 format job (pjh_platform .github/workflows/ci.yml)

```
1 format:
2   name: clang-format
3   runs-on: ubuntu-latest
4   steps:
5     - uses: actions/checkout@v4
6       with: { fetch-depth: 1 }
7     - run: pipx install clang-format==22.1.8
8     - run: clang-format --dry-run --Werror $(git ls-files '*.cpp' '*.hpp' '*.h')
```

🔗 本地复现（等价于 CI 的 Check formatting）

```
1 $ files=$(git ls-files '*.cpp' '*.hpp' '*.h')
2 $ clang-format --dry-run --Werror $files      # 干净树：无输出, exit 0
3 $ printf '\nint      broken    =1;\n' >> src/clock.cpp
4 $ clang-format --dry-run --Werror src/clock.cpp
5 src/clock.cpp:107:4: error: code should be clang-formatted [-Wclang-format-violations
6   ]
6 exit=1
```

✅ 结果与验证

- 干净分支：master run 34814274650 全绿（format job 6s + 6 档 build/test）；
- 故意弄坏：分支 format-violation-demo run 34814518490 中 clang-format 的 Check formatting 步骤 6s 内退出 1，而其余 6 个 build/test 矩阵全过——格式门禁能独立拦下违规。

https://github.com/pjh456/pjh_platform/actions/runs/34814518490

附带发现：原 push.branches: ["*"] 中的 * 不匹配含 / 的分支（如 fix/xxx 不会触发 CI），应改为 "**"。

💡 要点与注意事项

接入前先用 `clang-format -i` 把唯一不合规的 `tests/test_encoding.cpp` 规整，保证门禁基线全绿；CI 用 `pipx` 固定 `clang-format==22.1.8` 与本地版本一致，避免版本差异造成误报。题目要求的 `format/lint/test` 中，`build+test` 已由原矩阵覆盖，本次补上 `format`。

代码质量·4 用正则查找危险的 `shell=True` 调用

📖 题目要求

写一个正则并用 `grep` 在代码中查找 `subprocess.Popen(..., shell=True)`，再试着“破坏”这个正则；对比 `semgrep` 是否仍能匹配到那些让 `grep` 失效的危险代码。

🔗 正则 (`homework/code-quality/4.sh`)

```
1 grep -rnp 'subprocess\.Popen\[^\]*shell\s*=\s*True' .
```

✅ 结果与验证

```
$ sh 4.sh
./popen_samples.py:3: subprocess.Popen(cmd, shell=True)
./popen_samples.py:8: subprocess.Popen(cmd, shell = True)
./popen_samples.py:9: subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE)
./popen_samples.py:12: # 注释里的 ... (误报)
./popen_samples.py:13: text = "..." (误报)
```

`grep` 漏掉跨行的第 4-7 行调用，并误报注释与字符串；`semgrep -e 'subprocess.Popen(..., shell=True, ...)'` 得到 4 个 finding，正确包含跨行那条、且不命中注释与字符串。

💡 要点与注意事项

`[^)]*` 不匹配换行，所以跨行调用必漏；参数再嵌套括号时还会提前结束。语义检测（AST 级）的 `semgrep` 不受换行/空格影响，也不会把注释、字符串当代码，更适合这类危险调用扫描。

代码质量·5 用编辑器正则替换 Markdown 列表符

📖 题目要求

在 IDE/编辑器中用正则查找替换，把讲义中的 - Markdown 项目符号替换为 *；注意不能直接替换文件里所有 -。

</> 正则 (homework/code-quality/5.txt)

```

1 Vim:      :%s/\v^(\s*)- /\1* /g
2 VS Code:  查找 ^(\s*)-      替换 $1*
3 sed -E:   sed -E 's/^\([^[:space:]]*\)- /\1* /'

```

✓ 结果与验证

```

$ rg -c '^-' code-quality.md
28
$ sed -E 's/^\([^[:space:]]*\)- /\1* /' code-quality.md | diff - code-quality.md |
  grep -c '^>'
28
- [Formatting](#formatting)    -> * [Formatting](#formatting)
- [Linting](#linting)         -> * [Linting](#linting)

```

28 个行首项目符号全部替换；链接锚点 (#continuous-integration)、代码中的 - 等不受影响。

💡 要点与注意事项

关键是锚定行首并回填缩进: `^(\s*)-` 只匹配“行首（可含空白）+ - + 一个空格”，因此不会动锚点、`--flag`、`a-b` 等行内 -。

代码质量·6 用正则捕获 JSON 中的 name**📖 题目要求**

写正则从 `{"name": "Alyssa P. Hacker", "college": "MIT"}` 中捕获 name；处理 name 含转义 \ 的情形；再用 JSON 解析器写一个从 stdin 读、向 stdout 输出 name 的命令行程序。

</> 正则与结果 (homework/code-quality/6.txt、6.py)

```

1 "name"\s*:\s*"(.+)"           # 贪婪，多吞后面字段
2 "name"\s*:\s*"([^"]*)"        # 排除引号，正确
3 "name"\s*:\s*"((?:[\"\\]|\\.)*)" # 再跳过被转义的引号

```

✓ 结果与验证

```

greedy      : Alyssa P. Hacker", "college": "MIT
non-greedy: Alyssa P. Hacker
escaped     : Alyssa \"P.\" Hacker
$ echo '{"name": "Alyssa P. Hacker", "college": "MIT"}' | python3 6.py
Alyssa P. Hacker

```

💡 要点与注意事项

· 默认不匹配换行、.+ 贪婪会吃到行尾最后一个引号；`[^"]*` 用“排除引号”替代懒惰匹配更直观；实践中解析 JSON 应用 `json` 解析器，正则只适合极简、格式受控的抽取。

四 课后练习：missing-semester 《不止于代码》**不止于代码·2 好提交信息与弱提交信息的差距****📖 题目要求**

查看一个项目的提交历史，找到一条能解释“为什么要改”的好提交信息，再找一条只描述“改了什么”的弱提交信息；对后者查看 diff，尝试按“问题 → 方案 → 影响”结构写一条更好的提交信息，体会事后补全背景信息的成本。

🔗 好例子 (LMCache-Ascend, 我贡献的 PR #289 / #288)

```
3091f21 fix(c_ops): add missing <stdexcept> include in mem_alloc.cpp
      std::runtime_error is used but <stdexcept> was only pulled in
      transitively; GCC 13.3 / newer libstdc++ no longer provides it, so
      compilation fails with 'runtime_error' is not a member of 'std'.

fe407e0 fix(hixl): add pkg_inc include path for CANN 9.x
      rt_external_device.h 迁移到 aarch64-linux/pkg_inc/runtime; 新增
      -I 在旧 CANN (8.5) 上无害。
```

🔗 弱例子 (NovaShen555/WeatherViewer, 历史提交)

241eebfa 补一下	正文：忘记传templates了好像
7b9ee325 修了一下extends继承	正文：把tmp模板修好了

🔗 重写 241eebfa (依据 diff: 补传 11 个模板、6000+ 行)

`fix(templates)`: 补传遗漏的页面模板, 恢复页面渲染

问题：上一次提交只更新了视图，漏提交 `templates/` 下页面模板，部署后对应页面会因找不到模板而无法渲染。

方案：补传 `bulletin.html`、`home.html`、`toolmap.html` 等 11 个模板，并同步更新 `view_weather.html` 等引用。

影响：页面恢复渲染；本次变更含大段重排，需回归验证首页、气象公报、天气地图三类页面的渲染与静态资源路径。

📌 结果与验证

好信息把“为什么”写进正文（传递 include 的脆弱、旧版本兼容性）；弱信息只有“当时的操作”（补一下/修一下），几个月后除 diff 外无法还原动机与范围。

💡 要点与注意事项

按“问题 → 方案 → 影响”重写时，最费劲的正是补全“为什么”——这恰好说明提交信息应该在改动当下写，而不是事后来考古。

不止于代码·6 读懂一次真实 PR 评审

📖 题目要求

找一个已合并、且包含实质性评审意见（不只是 LGTM）的 PR，通读整个评审：所有评论是否同样高效？如果你是作者，收到这些评论体验如何？

🔗 评审线程（LMCache-Ascend #289）

1. chloroethylene: "LGTM — confirmed the bug is real 📌"
逐层验证根因（9 处 `std::runtime_error` 无 `<stdexcept>`；GCC 13 从 `bits/std_mutex.h` 移除 `<system_error>`），并建议 `pybind.cpp` 同类问题可顺带修或另开 PR，明确 "either way this one is ready to merge"。
2. pjh456: 已在 73c3da7 一并修复 `pybind.cpp` / `exception.h` 等 4 个文件。
3. chloroethylene: "fix lint plz"
4. pjh456: 1736ffc 修好 `clang-format` include 排序，CI 应能通过。
5. matthewygf: APPROVED, "LGTM ! Thanks for the contribution."

📌 结果与验证

评论 1 高质量：验证根因、给出可操作建议、并区分“建议/阻断”，作者无需来回确认。评论 3 是必要阻断项但信息量低（未指出是 `clang-format`、哪个文件、哪条规则），作者需自行翻 CI 日志。作为作者整体体验正面：“验证 + 可操作 + 分级”的反馈让作者知道该做什么。

💡 要点与注意事项

更高效的阻断意见应带上下文，例如：“`clang-format` 未通过：`pybind.cpp` 的 `<stdexcept>` 需按字母序排在 `<pybind11/stl.h>` 之后”。评审的价值在于让作者少猜。

五 总结与收获

- 课堂实验（q13–q16）

- 质量门禁 = `ruff format --check + ruff check + pytest`, 用 `check.sh` 串起并保证退出码;
- Make 的依赖必须写全, `all/clean` 声明 `.PHONY`, 才能真正“只重建受影响的产物”;
- `curl -fsS + jq select/sort_by` 能把本地 HTTP 数据转成 Markdown 报告;
- wheel 的 SHA-256 是本次产物的指纹, 构建非字节可复现, 重建哈希会变。

• 课后练习 (missing-semester 三讲)

- 版本控制: `git filter-repo --invert-paths` 重写历史; `stash` 存在 `refs/stash`; 别名与全局 `gitignore` 都可通过全局配置 (尤其 `~/.config/git/config`) 纳入 `dot-files` 管理;
- 代码质量: CI 用 `clang-format --dry-run --Werror` 做格式门禁并故意弄坏验证; `grep` 正则则在跨行/嵌套/注释上失效, `semgrep` 的 AST 匹配更可靠; 编辑器正则替换要锚定行首并回填缩进;
- 不止于代码: 提交信息要写“为什么”, 按“问题 → 方案 → 影响”组织; 好的评审 = 验证 + 可操作 + 分级 (阻断/建议)。